2650026_lab4

November 3, 2026

Roll No.	Subject Code	Assignment No.
2650026	CS69101	Lab 4

Given a set of integer elements stored in an input file, `input.txt`, construct a Red-Black Tree by inserting the elements one by one. Implement insertion and deletion operations while maintaining all the Red-Black Tree properties. Store the final tree contents in `output.txt` using in-order traversal.

[1]:
```
import os
import sys
from io import TextIOWrapper
from typing import Self
import random
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pylab import figure

sys.setrecursionlimit(100000) # Fix to avoid RecursionError

prep_arr = [random.randint(1, (i+1)*10) for i in range(10000)] # Random Array␣
 ↪of 10000 Elements
# prep_arr.sort() # Sorted Array for Worst Case Scenario
print(prep_arr[:25])
print(len(prep_arr))

# Write the array to a file
f = open("./2650026_lab4_data/input0.txt", "w")
for i in prep_arr:
    f.write(f"{i}, ")
f.write("\n")
f.close()

del prep_arr
del f

iterations = 0
PRINT_DEBUG = False
```

```
[6, 13, 3, 38, 35, 49, 19, 7, 71, 30, 8, 9, 125, 120, 4, 144, 120, 58, 169, 84,
194, 49, 152, 169, 119]
10000
```

1 Q1 & Q2

Read integer contents from `input.txt` and construct a Red-Black Tree using the `insert(key)` operation.

Implement Red-Black Tree insertion, including the required recolouring and LL, RR, LR, and RL rotations to maintain the Red-Black Tree properties.

1.1 File Read

[2]:
```
# Read a set of integer elements from an input file (input.txt)
arr0 = []
data0 = open("./2650026_lab4_data/input.txt", 'r')

try:
    for line in data0.readlines():
        line_data = line.rstrip().rstrip(',').replace(' ','').split(',') #using␣
 ↪rstrip to remove the \n
        for data in line_data:
            arr0.append(int(data))
    data0.close()
except Exception as e:
    sys.exit("[ERROR] Invalid data" + str(e))

print(f"Read {len(arr0)} elements.")
```

```
Read 17 elements.
```

[3]:
```
print(arr0)
```

```
[70, 50, 90, 77, 55, 348, 76, 67, 69, 420, 25, 911, 210, 108, 768, 1080, 720]
```

1.2 Red Black Tree

[4]:
```
class RB_Node:
    def __init__(self, value: int | None = None) -> None:
        self.__value = value
        self.__lchild = None
        self.__rchild = None
        self.__parent = None
        self.__color = '#F00' # Color in #RGB, new node is always red

    def __get_lchild(self) -> Self | None:
        return self.__lchild
```

```
    def __set_lchild(self, lchild: Self | None) -> None: # type: ignore
        if PRINT_DEBUG:
            print(f"Setting left child of node with value {self.__value} to␣
↪{lchild}")
        self.__lchild = lchild
        if lchild is not None:
            lchild.__parent = self

    lchild = property(__get_lchild, __set_lchild, doc="Left child of the node")

    def __get_rchild(self) -> Self | None:
        return self.__rchild

    def __set_rchild(self, rchild: Self | None) -> None: # type: ignore
        if PRINT_DEBUG:
            print(f"Setting right child of node with value {self.__value} to␣
↪{rchild}")
        self.__rchild = rchild
        if rchild is not None:
            rchild.__parent = self

    rchild = property(__get_rchild, __set_rchild, doc="Right child of the node")

    def __get_parent(self) -> Self | None:
        return self.__parent

    def __set_parent(self, parent: Self | None) -> None: # type: ignore
        if PRINT_DEBUG:
            print(f"Setting parent of node with value {self.__value} to␣
↪{parent}")
        self.__parent = parent

    parent = property(__get_parent, __set_parent, doc="Parent of the node")

    def __get_value(self) -> int | None:
        return self.__value

    def __set_value(self, value: int) -> None:
        self.__value = value

    value = property(__get_value, __set_value, doc="Value of the node")

    def __get_color(self) -> str | None:
        return self.__color

    def __set_color(self, color: str) -> None:
        self.__color = color
```

```
    color = property(__get_color, __set_color, doc="Color of the node")

    def __str__(self) -> str:
        return f"Node(value={self.__value}, color={self.__color}, lchild={self.
 ↪__lchild.value if self.__lchild else None}, rchild={self.__rchild.value if␣
 ↪self.__rchild else None}, parent={self.__parent.value if self.__parent else␣
 ↪None})"

    def __repr__(self) -> str:
        return self.__str__()
```

[ ]:
```
# def RB_right_rotate(k2_node: RB_Node | None, root_node: RB_Node | None) ->␣
 ↪RB_Node | None:
#     global iterations
#     if PRINT_DEBUG:
#         print(f"Right Rotate: k2_node={k2_node}, root_node={root_node}")
#     k1_node = k2_node.lchild if k2_node is not None else None
#     if k1_node is None or k2_node is None:
#         return root_node

#     iterations += 1
#     if PRINT_DEBUG:
#         print(f"LL Case: k1_node={k1_node}, k2_node={k2_node}")

#     # From Weiss' Data Structures and Algorithm Analysis in C++ (Page 156)
#     # Combined CLRS Logic with Weiss' Logic to avoid bugs

#     # SAFETY CODE
#     TEMP_NODE = k1_node.rchild
#     OG_PARENT = k2_node.parent
#     WAS_RCHILD = bool(OG_PARENT is not None and OG_PARENT.rchild is k2_node)

#     # CLRS: x.lchild = y.rchild
#     # CLRS: y.rchild = x
#     k1_node.rchild = k2_node
#     k2_node.lchild = TEMP_NODE

#     # Update Parents

#     # CLRS: if y.rchild is not None:
#     # CLRS:     y.rchild.parent = y
#     # CLRS: // However, y.rchild is now x.lchild, so we need to update x.
 ↪lchild's parent to y
#     if PRINT_DEBUG:
#         print(f"Updating parent of {k2_node.lchild} to {k2_node}")
#     if k2_node.lchild is not None:
```

```
#         k2_node.lchild.parent = k2_node

#     # CLRS: x.parent = y.parent
#     k1_node.parent = OG_PARENT

#     # CLRS: if y.parent is None:
#     # CLRS:     root_node = y
#     if OG_PARENT is None:
#         root_node = k1_node
#     # CLRS: elseif y is y.parent.rchild:
#     # CLRS:     y.parent.rchild = x
#     elif WAS_RCHILD:
#         OG_PARENT.rchild = k1_node
#         # OG_PARENT.update_height_and_balance() # Update height and balance␣
 ↪factor of OG_PARENT
#     # CLRS: else:
#     # CLRS:     y.parent.lchild = x
#     else:
#         OG_PARENT.lchild = k1_node
#         # OG_PARENT.update_height_and_balance() # Update height and balance␣
 ↪factor of OG_PARENT


#     # CLRS: y.parent = x
#     # k2_node.parent = k1_node (THIS IS AUTO HANDLED BY SETTER METHOD)

#     # if k1_node.rchild is not None:
#     #     k1_node.rchild.parent = k1_node

#     # root_node = k1_node if OG_PARENT is None else root_node // Copilot told␣
 ↪me this

#     # k1_node.update_height_and_balance() # Update height and balance factor␣
 ↪of k1_node
#     # k2_node.update_height_and_balance() # Update height and balance factor␣
 ↪of k2_node

#     if k1_node.parent is not None:
#         # k1_node.parent.update_height_and_balance() # Update height and␣
 ↪balance factor of k1_node's parent
#         pass

#     return root_node

# def RB_left_rotate(k1_node: RB_Node | None, root_node: RB_Node | None) ->␣
 ↪RB_Node | None:
```

```
#     global iterations
#     if PRINT_DEBUG:
#         print(f"Left Rotate: k1_node={k1_node}, root_node={root_node}")
#     k2_node = k1_node.rchild if k1_node is not None else None
#     if k1_node is None or k2_node is None:
#         return root_node

#     iterations += 1
#     if PRINT_DEBUG:
#         print(f"RR Case: k1_node={k1_node}, k2_node={k2_node}")

#     # From Weiss' Data Structures and Algorithm Analysis in C++ (Page 156)
#     # Combined CLRS Logic with Weiss' Logic to avoid bugs

#     # SAFETY CODE
#     TEMP_NODE = k2_node.lchild
#     OG_PARENT = k1_node.parent
#     WAS_RCHILD = bool(OG_PARENT is not None and OG_PARENT.rchild is k1_node)

#     # CLRS: x.rchild = y.lchild
#     # CLRS: y.lchild = x
#     k2_node.lchild = k1_node
#     k1_node.rchild = TEMP_NODE

#     # Update Parents

#     # CLRS: if y.lchild is not None:
#     # CLRS:     y.lchild.parent = x
#     # CLRS: // However, y.lchild is now x.rchild, so we need to update x.
 ↪rchild's parent to y
#     if k1_node.rchild is not None:
#         k1_node.rchild.parent = k1_node

#     # CLRS: y.parent = x.parent
#     k2_node.parent = OG_PARENT

#     # CLRS: if x.parent is None:
#     # CLRS:     root_node = y
#     if OG_PARENT is None:
#         root_node = k2_node
#     # CLRS: elseif x is x.parent.lchild:
#     # CLRS:     x.parent.lchild = y
#     elif WAS_RCHILD:
#         OG_PARENT.rchild = k2_node
#         # OG_PARENT.update_height_and_balance() # Update height and balance␣
 ↪factor of OG_PARENT
#     # CLRS: else:
```

```
#     # CLRS:     x.parent.rchild = y
#     else:
#         OG_PARENT.lchild = k2_node
#         # OG_PARENT.update_height_and_balance() # Update height and balance factor of OG_PARENT

#     # CLRS: x.parent = y
#     # k1_node.parent = k2_node (THIS IS AUTO HANDLED BY SETTER METHOD)

#     # k1_node.update_height_and_balance() # Update height and balance factor of k1_node
#     # k2_node.update_height_and_balance() # Update height and balance factor of k2_node

#     if k2_node.parent is not None:
#         # k2_node.parent.update_height_and_balance() # Update height and balance factor of k2_node's parent
#         pass

#     return root_node
```

[6]:
```
def RB_right_rotate(k2_node: RB_Node | None, root_node: RB_Node | None) -> RB_Node | None:
    global iterations
    if PRINT_DEBUG:
        print(f"Right Rotate: k2_node={k2_node}, root_node={root_node}")
    k1_node = k2_node.lchild if k2_node is not None else None
    if k1_node is None or k2_node is None:
        return root_node

    iterations += 1
    if PRINT_DEBUG:
        print(f"LL Case: k1_node={k1_node}, k2_node={k2_node}")

    # From Weiss' Data Structures and Algorithm Analysis in C++ (Page 156)
    # Combined CLRS Logic with Weiss' Logic to avoid bugs

    # SAFETY CODE
    TEMP_NODE = k1_node.rchild
    OG_PARENT = k2_node.parent
    WAS_RCHILD = bool(OG_PARENT is not None and OG_PARENT.rchild is k2_node)

    # CLRS: x.lchild = y.rchild
    # CLRS: y.rchild = x
    k1_node.rchild = k2_node
    k2_node.lchild = TEMP_NODE
```

```
    # Update Parents

    # CLRS: if y.rchild is not None:
    # CLRS:     y.rchild.parent = y
    # CLRS: // However, y.rchild is now x.lchild, so we need to update x.
 ↪lchild's parent to y
    if PRINT_DEBUG:
        print(f"Updating parent of {k2_node.lchild} to {k2_node}")
    if k2_node.lchild is not None:
        k2_node.lchild.parent = k2_node

    # CLRS: x.parent = y.parent
    k1_node.parent = OG_PARENT

    # CLRS: if y.parent is None:
    # CLRS:     root_node = y
    if OG_PARENT is None:
        root_node = k1_node
    # CLRS: elseif y is y.parent.rchild:
    # CLRS:     y.parent.rchild = x
    elif WAS_RCHILD:
        OG_PARENT.rchild = k1_node
    # CLRS: else:
    # CLRS:     y.parent.lchild = x
    else:
        OG_PARENT.lchild = k1_node


    # CLRS: y.parent = x
    # k2_node.parent = k1_node (THIS IS AUTO HANDLED BY SETTER METHOD)


    return root_node
```

[7]:
```
def RB_left_rotate(k1_node: RB_Node | None, root_node: RB_Node | None) ->␣
 ↪RB_Node | None:
    global iterations
    if PRINT_DEBUG:
        print(f"Left Rotate: k1_node={k1_node}, root_node={root_node}")
    k2_node = k1_node.rchild if k1_node is not None else None
    if k1_node is None or k2_node is None:
        return root_node

    iterations += 1
    if PRINT_DEBUG:
        print(f"RR Case: k1_node={k1_node}, k2_node={k2_node}")
```

```
    # From Weiss' Data Structures and Algorithm Analysis in C++ (Page 156)
    # Combined CLRS Logic with Weiss' Logic to avoid bugs

    # SAFETY CODE
    TEMP_NODE = k2_node.lchild
    OG_PARENT = k1_node.parent
    WAS_RCHILD = bool(OG_PARENT is not None and OG_PARENT.rchild is k1_node)

    # CLRS: x.rchild = y.lchild
    # CLRS: y.lchild = x
    k2_node.lchild = k1_node
    k1_node.rchild = TEMP_NODE

    # Update Parents

    # CLRS: if y.lchild is not None:
    # CLRS:     y.lchild.parent = x
    # CLRS: // However, y.lchild is now x.rchild, so we need to update x.
↪rchild's parent to y
    if k1_node.rchild is not None:
        k1_node.rchild.parent = k1_node

    # CLRS: y.parent = x.parent
    k2_node.parent = OG_PARENT

    # CLRS: if x.parent is None:
    # CLRS:     root_node = y
    if OG_PARENT is None:
        root_node = k2_node
    # CLRS: elseif x is x.parent.lchild:
    # CLRS:     x.parent.lchild = y
    elif WAS_RCHILD:
        OG_PARENT.rchild = k2_node
    # CLRS: else:
    # CLRS:     x.parent.rchild = y
    else:
        OG_PARENT.lchild = k2_node

    # CLRS: x.parent = y
    # k1_node.parent = k2_node (THIS IS AUTO HANDLED BY SETTER METHOD)


    return root_node
```

[8]:
```
def RB_Insert_Fixup(root: RB_Node, Z_Node: RB_Node) -> RB_Node | None:
    global iterations
    while Z_Node.parent is not None and Z_Node.parent.color == '#F00':
```

```
        iterations += 1
        if Z_Node.parent == Z_Node.parent.parent.lchild:
            Y_Node = Z_Node.parent.parent.rchild
            if Y_Node is not None and Y_Node.color == '#F00':
                Z_Node.parent.color = '#000'
                Y_Node.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                Z_Node = Z_Node.parent.parent
            else:
                if Z_Node == Z_Node.parent.rchild:
                    Z_Node = Z_Node.parent
                    root = RB_left_rotate(Z_Node, root) # type: ignore
                Z_Node.parent.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                root = RB_right_rotate(Z_Node.parent.parent, root) # type:␣
 ↪ignore
        else:
            Y_Node = Z_Node.parent.parent.lchild
            if Y_Node is not None and Y_Node.color == '#F00':
                Z_Node.parent.color = '#000'
                Y_Node.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                Z_Node = Z_Node.parent.parent
            else:
                if Z_Node == Z_Node.parent.lchild:
                    Z_Node = Z_Node.parent
                    root = RB_right_rotate(Z_Node, root) # type: ignore
                Z_Node.parent.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                root = RB_left_rotate(Z_Node.parent.parent, root) # type: ignore
    root.color = '#000'
    return root

def RB_Insert(root: RB_Node | None, value: int) -> RB_Node | None:
    global iterations
    Z_Node = RB_Node(value)

    Y_NODE = None
    X_NODE = root
    while X_NODE is not None:
        iterations += 1
        Y_NODE = X_NODE
        if value < X_NODE.value:
            X_NODE = X_NODE.lchild
        else:
            X_NODE = X_NODE.rchild
```

```
    Z_Node.parent = Y_NODE

    if Y_NODE is None:
        root = Z_Node
    elif value < Y_NODE.value:
        Y_NODE.lchild = Z_Node
    else:
        Y_NODE.rchild = Z_Node

    # New Node's Color is already RED by default.

    root = RB_Insert_Fixup(root, Z_Node) # type: ignore
    return root
```

[9]:
```
def RB_inorder_traversal(root: RB_Node | None, out_file: TextIOWrapper | None)␣
 ↪-> None:
    if root is None:
        return
    RB_inorder_traversal(root.lchild, out_file = out_file)
    if out_file is not None:
        print(root.value, end = ', ', file = out_file)
    print(root.value, end = ', ')
    print(root)
    RB_inorder_traversal(root.rchild, out_file = out_file)

def RB_preorder_traversal(root: RB_Node | None, out_file: TextIOWrapper | None)␣
 ↪-> None:
    if root is None:
        return
    if out_file is not None:
        print(root.value, end = ', ', file = out_file)
    print(root.value, end = ', ')
    print(root)
    RB_preorder_traversal(root.lchild, out_file)
    RB_preorder_traversal(root.rchild, out_file)

def RB_postorder_traversal(root: RB_Node | None, out_file: TextIOWrapper |␣
 ↪None) -> None:
    if root is None:
        return
    RB_postorder_traversal(root.lchild, out_file)
    RB_postorder_traversal(root.rchild, out_file)
    if out_file is not None:
        print(root.value, end = ', ', file = out_file)
    print(root.value, end=', ')
    print(root)
```

[10]:
```
root0 = None
for value in arr0:
    root0 = RB_Insert(root0, value)
```

[11]:
```
RB_inorder_traversal(root0, out_file = None)
```

```
25, Node(value=25, color=#F00, lchild=None, rchild=None, parent=50)
50, Node(value=50, color=#000, lchild=25, rchild=None, parent=55)
55, Node(value=55, color=#000, lchild=50, rchild=67, parent=70)
67, Node(value=67, color=#000, lchild=None, rchild=69, parent=55)
69, Node(value=69, color=#F00, lchild=None, rchild=None, parent=67)
70, Node(value=70, color=#000, lchild=55, rchild=420, parent=None)
76, Node(value=76, color=#F00, lchild=None, rchild=None, parent=77)
77, Node(value=77, color=#000, lchild=76, rchild=None, parent=90)
90, Node(value=90, color=#F00, lchild=77, rchild=210, parent=420)
108, Node(value=108, color=#F00, lchild=None, rchild=None, parent=210)
210, Node(value=210, color=#000, lchild=108, rchild=348, parent=90)
348, Node(value=348, color=#F00, lchild=None, rchild=None, parent=210)
420, Node(value=420, color=#000, lchild=90, rchild=911, parent=70)
720, Node(value=720, color=#F00, lchild=None, rchild=None, parent=768)
768, Node(value=768, color=#000, lchild=720, rchild=None, parent=911)
911, Node(value=911, color=#F00, lchild=768, rchild=1080, parent=420)
1080, Node(value=1080, color=#000, lchild=None, rchild=None, parent=911)
```

Roll No.	Subject Code	Assignment No.
2650026	CS69101	Lab 5

2 Q3.

Implement delete(key) to remove a specified element from the tree and perform the required re-colouring and rotations/fix-up operations.

2.1 Deletion

[12]:
```
def RB_Search(root: RB_Node | None, key: int) -> RB_Node | None:
    global iterations
    X_NODE = root
    while X_NODE is not None:
        iterations += 1
        if key == X_NODE.value:
            return X_NODE
        X_NODE = X_NODE.lchild if key < X_NODE.value else X_NODE.rchild
    return None


def RB_Minimum(node: RB_Node) -> RB_Node:
    global iterations
```

```
    while node.lchild is not None:
        iterations += 1
        node = node.lchild
    return node


def RB_is_black(node: RB_Node | None) -> bool:
    # A missing (None) child is a black NIL leaf
    return node is None or node.color == '#000'


def RB_Transplant(root: RB_Node | None, u: RB_Node, v: RB_Node | None) ->␣
↪RB_Node | None:
    global iterations
    iterations += 1
    # CLRS RB-TRANSPLANT: replace the subtree rooted at u with the subtree␣
↪rooted at v
    if u.parent is None:
        root = v
        if v is not None:
            v.parent = None
    elif u is u.parent.lchild:
        u.parent.lchild = v   # setter also sets v.parent
    else:
        u.parent.rchild = v   # setter also sets v.parent
    return root
```

[13]:
```
def RB_Delete_Fixup(root: RB_Node | None, x: RB_Node | None, x_parent: RB_Node␣
↪| None) -> RB_Node | None:
    global iterations
    # x carries the extra black. x may be None (NIL), so its parent is tracked␣
↪separately.
    while x is not root and RB_is_black(x):
        if x_parent is None:
            break
        iterations += 1
        if x is x_parent.lchild:
            W_Node = x_parent.rchild
            if W_Node is not None and W_Node.color == '#F00':        # Case 1:␣
↪sibling red
                W_Node.color = '#000'
                x_parent.color = '#F00'
                root = RB_left_rotate(x_parent, root)
                W_Node = x_parent.rchild
            if RB_is_black(W_Node.lchild if W_Node is not None else None) and␣
↪RB_is_black(W_Node.rchild if W_Node is not None else None):
```

```
                # Case 2: both nephews black (a missing sibling is treated as␣
↪black)
                if W_Node is not None:
                    W_Node.color = '#F00'
                x = x_parent
                x_parent = x.parent
            else:
                if RB_is_black(W_Node.rchild if W_Node is not None else None): ␣
↪# Case 3: far nephew black
                    if W_Node is not None and W_Node.lchild is not None:
                        W_Node.lchild.color = '#000'
                    if W_Node is not None:
                        W_Node.color = '#F00'
                        root = RB_right_rotate(W_Node, root)
                    W_Node = x_parent.rchild
                if W_Node is not None:
                    W_Node.color = x_parent.color                              #␣
↪Case 4: far nephew red
                    x_parent.color = '#000'
                    if W_Node.rchild is not None:
                        W_Node.rchild.color = '#000'
                root = RB_left_rotate(x_parent, root)
                x = root
                x_parent = None
        else:                                                        # mirror␣
↪image
            W_Node = x_parent.lchild
            if W_Node is not None and W_Node.color == '#F00':
                W_Node.color = '#000'
                x_parent.color = '#F00'
                root = RB_right_rotate(x_parent, root)
                W_Node = x_parent.lchild
            if RB_is_black(W_Node.lchild if W_Node is not None else None) and␣
↪RB_is_black(W_Node.rchild if W_Node is not None else None):
                if W_Node is not None:
                    W_Node.color = '#F00'
                x = x_parent
                x_parent = x.parent
            else:
                if RB_is_black(W_Node.lchild if W_Node is not None else None):
                    if W_Node is not None and W_Node.rchild is not None:
                        W_Node.rchild.color = '#000'
                    if W_Node is not None:
                        W_Node.color = '#F00'
                        root = RB_left_rotate(W_Node, root)
                    W_Node = x_parent.lchild
                if W_Node is not None:
```

```
                    W_Node.color = x_parent.color
                    x_parent.color = '#000'
                    if W_Node.lchild is not None:
                        W_Node.lchild.color = '#000'
                root = RB_right_rotate(x_parent, root)
                x = root
                x_parent = None
    if x is not None:
        x.color = '#000'
    return root


def RB_Delete(root: RB_Node | None, key: int) -> RB_Node | None:
    Z_Node = RB_Search(root, key)
    if Z_Node is None:
        print(f"[WARN] Key {key} not found, nothing deleted")
        return root

    Y_Node = Z_Node
    Y_ORIG_COLOR = Y_Node.color

    if Z_Node.lchild is None:                    # 0 or 1 child (right)
        X_Node = Z_Node.rchild
        X_Parent = Z_Node.parent
        root = RB_Transplant(root, Z_Node, Z_Node.rchild)
    elif Z_Node.rchild is None:                  # 1 child (left)
        X_Node = Z_Node.lchild
        X_Parent = Z_Node.parent
        root = RB_Transplant(root, Z_Node, Z_Node.lchild)
    else:                                        # 2 children: use in-order␣
 ↪successor
        Y_Node = RB_Minimum(Z_Node.rchild)
        Y_ORIG_COLOR = Y_Node.color
        X_Node = Y_Node.rchild
        if Y_Node.parent is Z_Node:
            X_Parent = Y_Node
        else:
            X_Parent = Y_Node.parent
            root = RB_Transplant(root, Y_Node, Y_Node.rchild)
            Y_Node.rchild = Z_Node.rchild
        root = RB_Transplant(root, Z_Node, Y_Node)
        Y_Node.lchild = Z_Node.lchild
        Y_Node.color = Z_Node.color

    # Keep the NIL child's parent valid for the fix-up when it exists.
    if X_Node is not None:
        X_Parent = X_Node.parent
```

```
    # Detach the removed node
    Z_Node.lchild = None
    Z_Node.rchild = None
    Z_Node.parent = None

    if Y_ORIG_COLOR == '#000':                    # only a removed black node␣
 ↪breaks black-height
        root = RB_Delete_Fixup(root, X_Node, X_Parent)
    if root is not None:
        root.parent = None
    return root
```

[14]:
```
def RB_Verify(root: RB_Node | None) -> int:
    # Returns node count; raises AssertionError if any Red-Black property is␣
 ↪broken
    if root is None:
        return 0
    assert root.parent is None, "root has a parent"
    assert root.color == '#000', "root is not black"
    def rec(n, lo, hi):
        if n is None:
            return 0, 1
        assert lo <= n.value <= hi, f"BST order broken at {n.value}"
        assert n.color in ('#000', '#F00'), f"invalid color at {n.value}"
        if n.color == '#F00':
            assert RB_is_black(n.lchild) and RB_is_black(n.rchild), f"red-red␣
 ↪at {n.value}"
        for c in (n.lchild, n.rchild):
            assert c is None or c.parent is n, f"bad parent link under {n.
 ↪value}"
        cl, bl = rec(n.lchild, lo, n.value)
        cr, br = rec(n.rchild, n.value, hi)
        assert bl == br, f"black-height mismatch at {n.value}"
        return cl + cr + 1, bl + (1 if n.color == '#000' else 0)
    return rec(root, float('-inf'), float('inf'))[0]


def RB_Height(root: RB_Node | None) -> int:
    if root is None:
        return 0
    return max(RB_Height(root.lchild), RB_Height(root.rchild)) + 1


def RB_Inorder_Values(root: RB_Node | None) -> list[int]:
    if root is None:
        return []
```

```
    return RB_Inorder_Values(root.lchild) + [root.value] +␣
↪RB_Inorder_Values(root.rchild)
```

[15]:
```
root0 = RB_Delete(root0, 420)
RB_Verify(root0)

with open("./2650026_lab4_data/output.txt", "w") as out_file:
    RB_inorder_traversal(root0, out_file = out_file)
    out_file.close()
```

```
25, Node(value=25, color=#F00, lchild=None, rchild=None, parent=50)
50, Node(value=50, color=#000, lchild=25, rchild=None, parent=55)
55, Node(value=55, color=#000, lchild=50, rchild=67, parent=70)
67, Node(value=67, color=#000, lchild=None, rchild=69, parent=55)
69, Node(value=69, color=#F00, lchild=None, rchild=None, parent=67)
70, Node(value=70, color=#000, lchild=55, rchild=720, parent=None)
76, Node(value=76, color=#F00, lchild=None, rchild=None, parent=77)
77, Node(value=77, color=#000, lchild=76, rchild=None, parent=90)
90, Node(value=90, color=#F00, lchild=77, rchild=210, parent=720)
108, Node(value=108, color=#F00, lchild=None, rchild=None, parent=210)
210, Node(value=210, color=#000, lchild=108, rchild=348, parent=90)
348, Node(value=348, color=#F00, lchild=None, rchild=None, parent=210)
720, Node(value=720, color=#000, lchild=90, rchild=911, parent=70)
768, Node(value=768, color=#000, lchild=None, rchild=None, parent=911)
911, Node(value=911, color=#F00, lchild=768, rchild=1080, parent=720)
1080, Node(value=1080, color=#000, lchild=None, rchild=None, parent=911)
```

3 Q6

For different numbers of nodes, such as 100, 500, 1000, 5000, 10000 and 50000, measure and plot:

- Tree Height vs. Number of Nodes
- Insertion Time vs. Number of Nodes
- Deletion Time vs. Number of Nodes
- Search Time vs. Number of Nodes

The operation cost is measured using iteration/computation counts. The 50,000-node case is optional.

[18]:
```
arr1 = []
with open("./2650026_lab4_data/input0.txt", 'r') as data0:
    for line in data0.readlines():
        line_data = line.rstrip().rstrip(',').replace(' ','').split(',')
        for data in line_data:
            arr1.append(int(data))
```

[19]:
```
SIZES = [100, 500, 1000, 5000, 10000]
N = SIZES[-1]
```

```
random.seed(911)

def RB_Build_Measured(values: list[int]):
    global iterations
    root = None
    insert_counts = []
    for value in values:
        iterations = 0
        root = RB_Insert(root, value)
        insert_counts.append(iterations)
    return root, insert_counts


def RB_Search_Measured(root: RB_Node | None, key: int):
    global iterations
    iterations = 0
    node = RB_Search(root, key)
    return node, iterations


def RB_Delete_Measured(root: RB_Node | None, key: int):
    global iterations
    iterations = 0
    root = RB_Delete(root, key)
    return root, iterations


def RB_Recorded(values: list[int], search_idx: list[int], delete_keys:␣
 ↪list[int]) -> dict:
    global iterations
    root = None
    height_store, insert_counts, search_counts, delete_counts = [], [], [], []
    for i, value in enumerate(values):
        iterations = 0
        root = RB_Insert(root, value)
        insert_counts.append(iterations)
        height_store.append(RB_Height(root))  # recorded once after every␣
 ↪insertion, as in lab 3
        search_counts.append(RB_Search_Measured(root, values[search_idx[i]])[1])
    RB_Verify(root)

    for key in delete_keys:
        root, count = RB_Delete_Measured(root, key)
        delete_counts.append(count)
    assert root is None, "Red-Black Tree is not empty after deleting every␣
 ↪element"
```

```
    # Deletions were recorded from the full tree downwards: reverse them so␣
 ↪that index j belongs to size j + 1
    return {"Height": height_store, "Insertion": insert_counts, "Search":␣
 ↪search_counts, "Deletion": delete_counts[::-1]}


def size_mean(series: list, n: int) -> float:
    # Mean of the values recorded for the last 10% of the sizes up to n␣
 ↪(smooths the per-size noise)
    window = series[n - max(1, n // 10) : n]
    return sum(window) / len(window)


assert N <= len(arr1), "n cannot exceed the length of arr1"
start_idx = random.randint(0, len(arr1) - N)
random_values = arr1[start_idx : start_idx + N]

search_idx = [random.randrange(i + 1) for i in range(N)]  # step i searches one␣
 ↪of the first i + 1 inserted elements
delete_keys = random_values[:]
random.shuffle(delete_keys)                                  # every element is␣
 ↪deleted once, in random order

records = {}
records[("Random", "Red-Black Tree")] = RB_Recorded(random_values, search_idx,␣
 ↪delete_keys)

rb = records[("Random", "Red-Black Tree")]
task6_df = pd.DataFrame({
    "Nodes": SIZES,
    "Height": [rb["Height"][n - 1] for n in SIZES],
    "Insertion Iterations": [size_mean(rb["Insertion"], n) for n in SIZES],
    "Deletion Iterations": [size_mean(rb["Deletion"], n) for n in SIZES],
    "Search Iterations": [size_mean(rb["Search"], n) for n in SIZES],
})
print("Height at each size; iterations are the mean over the last 10% of the␣
 ↪sizes up to that size")
task6_df
```

```
Height at each size; iterations are the mean over the last 10% of the sizes up
to that size
```

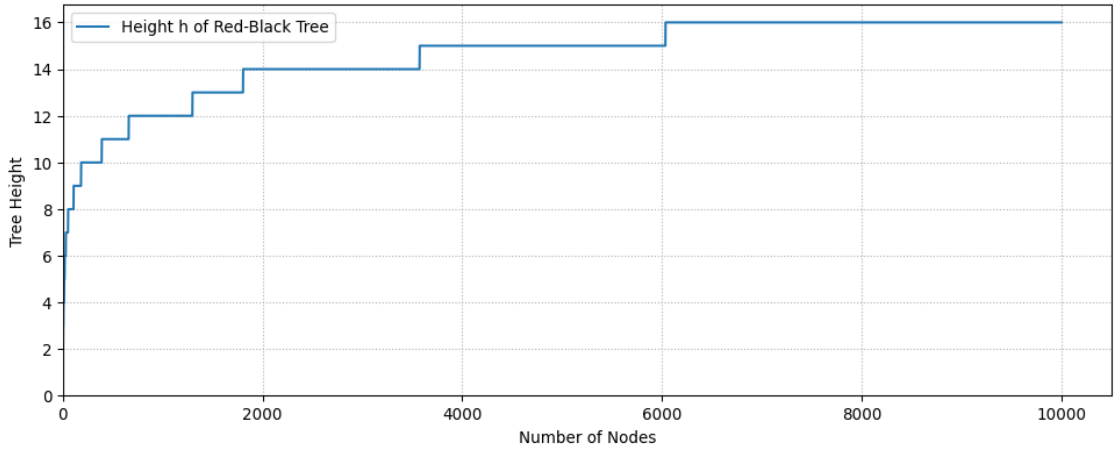
[19]:

	Nodes	Height	Insertion Iterations	Deletion Iterations	Search Iterations
0	100	8	8.100	8.500	5.900
1	500	11	10.660	11.020	7.600
2	1000	12	11.670	12.350	8.780

3	5000	15	14.214	14.410	11.376
4	10000	16	15.135	14.778	12.243

3.1 Height v/s Node Count

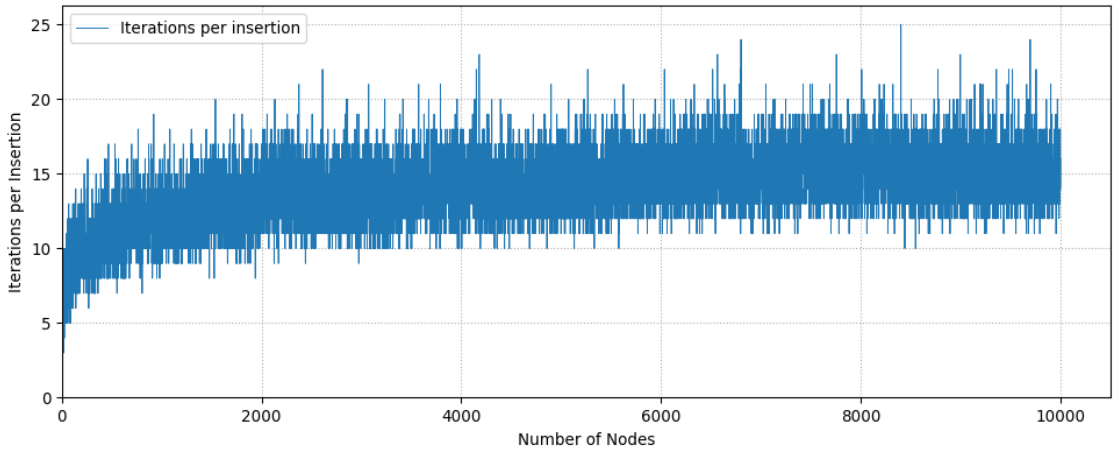
[21]:
```
figure(figsize=(12, 4.5), dpi=97)
y = records[("Random", "Red-Black Tree")]["Height"]
x = [i+1 for i in range(len(y))]
plt.plot(x, y, label="Height h of Red-Black Tree")
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.legend()
plt.xlabel("Number of Nodes")
plt.ylabel("Tree Height")
plt.grid(visible=True, linestyle=':')
plt.savefig("./2650026_lab4_data/task6_height.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



3.2 Iterations (Insertion) v/s Node Count

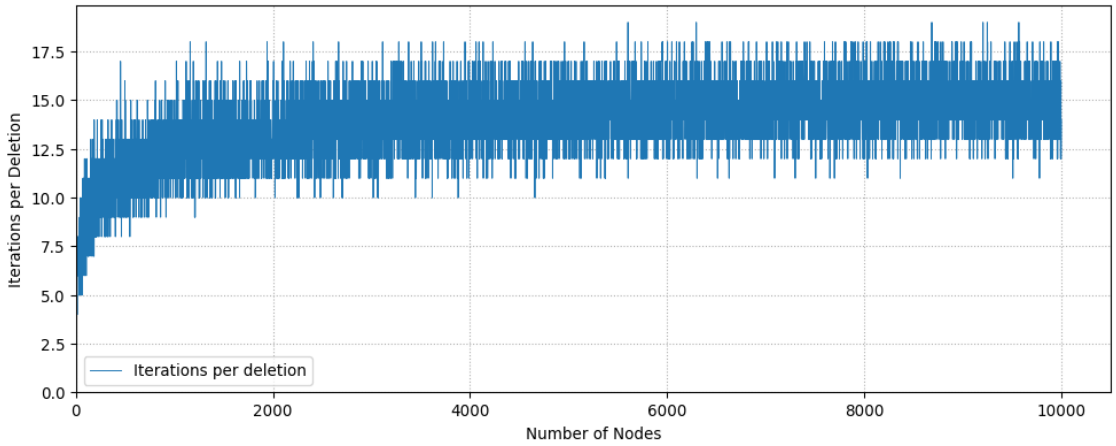
[22]:
```
figure(figsize=(12, 4.5), dpi=97)
y = records[("Random", "Red-Black Tree")]["Insertion"]
x = [i+1 for i in range(len(y))]
plt.plot(x, y, label="Iterations per insertion", linewidth=0.7)
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.legend()
plt.xlabel("Number of Nodes")
plt.ylabel("Iterations per Insertion")
plt.grid(visible=True, linestyle=':')
```

```
plt.savefig("./2650026_lab4_data/task6_insertion.png", dpi=150,␣
  ↪bbox_inches="tight")
plt.show()
```



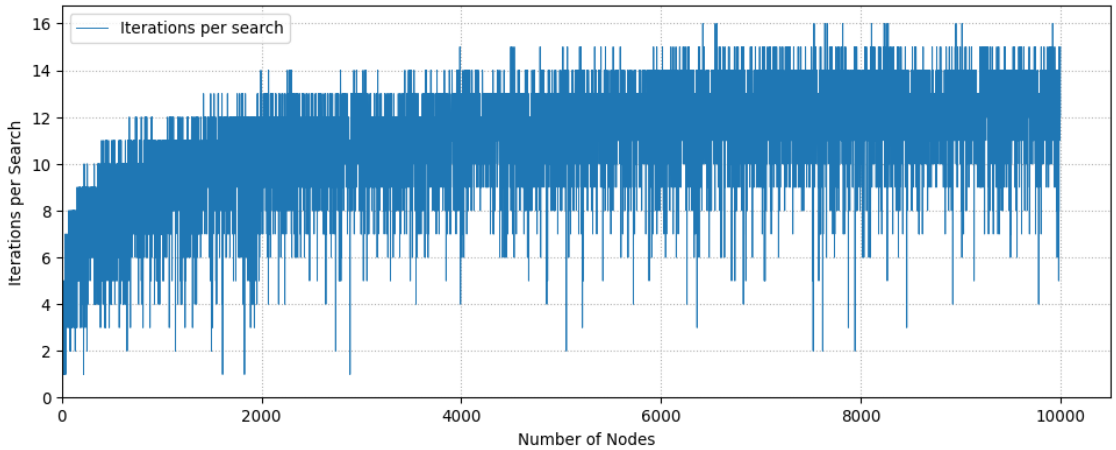
3.3 Iterations (Deletion) v/s Node Count

[23]:
```
figure(figsize=(12, 4.5), dpi=97)
y = records[("Random", "Red-Black Tree")]["Deletion"]
x = [i+1 for i in range(len(y))]
plt.plot(x, y, label="Iterations per deletion", linewidth=0.7)
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.legend()
plt.xlabel("Number of Nodes")
plt.ylabel("Iterations per Deletion")
plt.grid(visible=True, linestyle=':')
plt.savefig("./2650026_lab4_data/task6_deletion.png", dpi=150,␣
  ↪bbox_inches="tight")
plt.show()
```



3.4 Iterations (Search) v/s Node Count

[24]:
```
figure(figsize=(12, 4.5), dpi=97)
y = records[("Random", "Red-Black Tree")]["Search"]
x = [i+1 for i in range(len(y))]
plt.plot(x, y, label="Iterations per search", linewidth=0.7)
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.legend()
plt.xlabel("Number of Nodes")
plt.ylabel("Iterations per Search")
plt.grid(visible=True, linestyle=':')
plt.savefig("./2650026_lab4_data/task6_search.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



4 Q7

Implement an ordinary Binary Search Tree (BST) and experimentally compare its search time and height with the Red-Black Tree.

4.1 BST Tree: All Code

[25]:
```python
class BST_Node:
    def __init__(self):
        self.lchild = None
        self.rchild = None
        self.value = None
        self.parent = None


def BST_Insert(value, root_node: BST_Node | None = None) -> BST_Node:
    assert value is not None, "Value cannot be None"
    y = None
    x = root_node
    global iterations
    while x is not None:
        iterations += 1
        y = x
        if value < x.value:
            x = x.lchild
        else:
            x = x.rchild
    new_node = BST_Node()
    new_node.value = value
    new_node.parent = y
    if y is None:
        root_node = new_node
    elif value < y.value:
        y.lchild = new_node
    else:
        y.rchild = new_node
    return root_node


def BST_Search(value, root_node: BST_Node | None = None) -> BST_Node | None:
    global iterations
    if root_node is None:
        return None
    iterations += 1
    if value == root_node.value:
        return root_node
    elif value < root_node.value:
        return BST_Search(value, root_node.lchild)
```

```
    else:
        return BST_Search(value, root_node.rchild)


def BST_minimum(root_node: BST_Node | None = None) -> BST_Node | None:
    global iterations
    if root_node is None:
        return None
    while root_node.lchild is not None:
        iterations += 1
        root_node = root_node.lchild
    return root_node


def BST_Transplant(root_node: BST_Node | None = None, target_node: BST_Node |␣
↪None = None, replacement_node: BST_Node | None = None) -> BST_Node | None:
    global iterations
    iterations += 1
    if root_node is None:
        return None
    if target_node is None:
        return root_node
    if target_node.parent is None:
        root_node = replacement_node
    elif target_node == target_node.parent.lchild:
        target_node.parent.lchild = replacement_node
    else:
        target_node.parent.rchild = replacement_node
    if replacement_node is not None:
        replacement_node.parent = target_node.parent
    return root_node


def BST_Delete(root_node: BST_Node | None = None, target_node: BST_Node | None␣
↪= None) -> BST_Node | None:
    global iterations
    iterations += 1
    if root_node is None:
        return None
    if target_node is None:
        return root_node
    if target_node.lchild is None:
        root_node = BST_Transplant(root_node, target_node, target_node.rchild)
    elif target_node.rchild is None:
        root_node = BST_Transplant(root_node, target_node, target_node.lchild)
    else:
        successor_node = BST_minimum(target_node.rchild)
```

```
        if successor_node.parent != target_node:
            root_node = BST_Transplant(root_node, successor_node,␣
 ↪successor_node.rchild)
            successor_node.rchild = target_node.rchild
            successor_node.rchild.parent = successor_node
        root_node = BST_Transplant(root_node, target_node, successor_node)
        successor_node.lchild = target_node.lchild
        successor_node.lchild.parent = successor_node
    return root_node


def BST_Height(root_node: BST_Node | None = None) -> int:
    global iterations
    if root_node is None:
        return 0
    iterations += 1
    left_height = BST_Height(root_node.lchild)
    right_height = BST_Height(root_node.rchild)
    return max(left_height, right_height) + 1


def BST_Build_Measured(values: list[int]):
    global iterations
    root = None
    insert_counts = []
    for value in values:
        iterations = 0
        root = BST_Insert(value, root)
        insert_counts.append(iterations)
    return root, insert_counts


def BST_Search_Measured(root: BST_Node | None, key: int):
    global iterations
    iterations = 0
    node = BST_Search(key, root)
    return node, iterations


def BST_Delete_Measured(root: BST_Node | None, key: int):
    global iterations
    iterations = 0
    node = BST_Search(key, root)  # Include the search required to locate the␣
 ↪target
    root = BST_Delete(root, node)
    return root, iterations
```

```
def BST_Inorder_Values(root: BST_Node | None) -> list[int]:
    if root is None:
        return []
    return BST_Inorder_Values(root.lchild) + [root.value] +␣
 ↪BST_Inorder_Values(root.rchild)


def BST_Recorded(values: list[int], search_idx: list[int], delete_keys:␣
 ↪list[int]) -> dict:
    global iterations
    root = None
    height_store, insert_counts, search_counts, delete_counts = [], [], [], []
    for i, value in enumerate(values):
        iterations = 0
        root = BST_Insert(value, root)
        insert_counts.append(iterations)
        height_store.append(BST_Height(root))  # recorded once after every␣
 ↪insertion, as in lab 3
        search_counts.append(BST_Search_Measured(root,␣
 ↪values[search_idx[i]])[1])

    for key in delete_keys:
        root, count = BST_Delete_Measured(root, key)
        delete_counts.append(count)
    assert root is None, "BST is not empty after deleting every element"

    return {"Height": height_store, "Insertion": insert_counts, "Search":␣
 ↪search_counts, "Deletion": delete_counts[::-1]}
```

[ ]:
```
task7_rows = []
for n in SIZES:
    values = random.sample(range(1, n * 10 + 1), n)
    bst_root, _ = BST_Build_Measured(values)
    rb_root, _ = RB_Build_Measured(values)
    RB_Verify(rb_root)

    # Height calculation is performed separately from operation iteration␣
 ↪counts.
    bst_height = BST_Height(bst_root)
    rb_height = RB_Height(rb_root)
    search_keys = random.sample(values, min(100, n))
    bst_search = [BST_Search_Measured(bst_root, key)[1] for key in search_keys]
    rb_search = [RB_Search_Measured(rb_root, key)[1] for key in search_keys]

    task7_rows.append({
        "Nodes": n,
```

```
        "BST Height": bst_height,
        "Red-Black Tree Height": rb_height,
        "BST Average Search Iterations": sum(bst_search) / len(bst_search),
        "Red-Black Tree Average Search Iterations": sum(rb_search) /␣
 ↪len(rb_search),
    })

task7_df = pd.DataFrame(task7_rows)
task7_df
```

4.2 Height Comparison

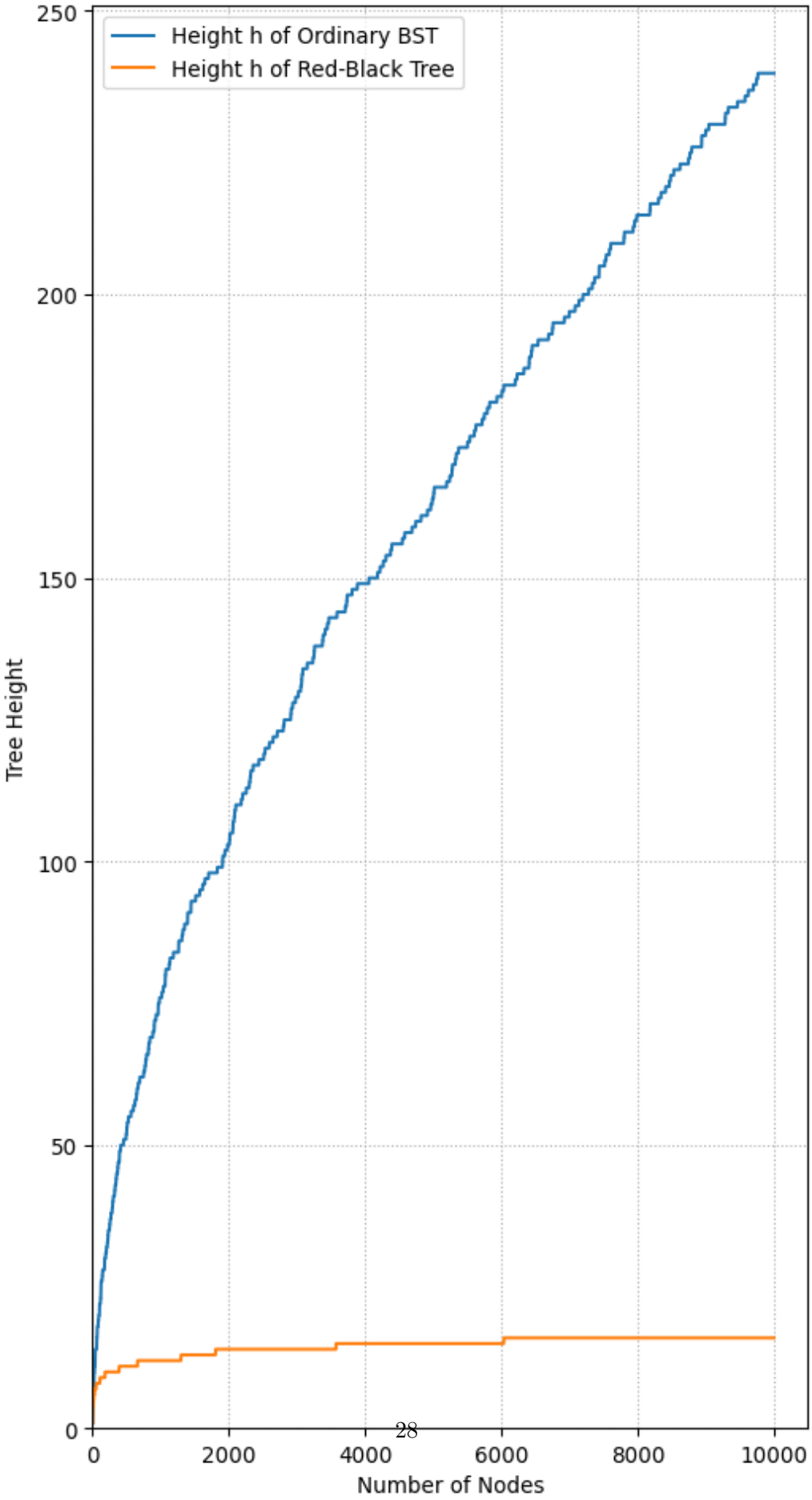
[26]:
```
records[("Random", "BST")] = BST_Recorded(random_values, search_idx,␣
 ↪delete_keys)

bst = records[("Random", "BST")]
task7_df = pd.DataFrame({
    "Nodes": SIZES,
    "BST Height": [bst["Height"][n - 1] for n in SIZES],
    "Red-Black Tree Height": [rb["Height"][n - 1] for n in SIZES],
})
task7_df
```

[26]:

	Nodes	BST Height	Red-Black Tree Height
0	100	20	8
1	500	52	11
2	1000	76	12
3	5000	164	15
4	10000	239	16

[29]:
```
figure(figsize=(6, 12), dpi=97)
y = records[("Random", "BST")]["Height"]
x = [i+1 for i in range(len(y))]
plt.plot(x, y, label="Height h of Ordinary BST")
y = records[("Random", "Red-Black Tree")]["Height"]
x = [i+1 for i in range(len(y))]
plt.plot(x, y, label="Height h of Red-Black Tree")
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.legend()
plt.xlabel("Number of Nodes")
plt.ylabel("Tree Height")
plt.grid(visible=True, linestyle=':')
plt.savefig("./2650026_lab4_data/task7_height.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```


Height h of Ordinary BST
Height h of Red-Black Tree
Tree Height
Number of Nodes
0
50
100
150
200
250
0
2000
4000
6000
8000
10000

4.3 Search Comparison

[30]:
```
task7_search_rows = []
for n in SIZES:
    assert n <= len(arr1), "n cannot exceed the length of arr1"
    start_idx = random.randint(0, len(arr1) - n)
    values = arr1[start_idx : start_idx + n]
    bst_root, _ = BST_Build_Measured(values)
    rb_root, _ = RB_Build_Measured(values)
    RB_Verify(rb_root)

    search_keys = random.sample(values, min(100, n))
    bst_search = [BST_Search_Measured(bst_root, key)[1] for key in search_keys]
    rb_search = [RB_Search_Measured(rb_root, key)[1] for key in search_keys]

    task7_search_rows.append({
        "Nodes": n,
        "BST Average Search Iterations": sum(bst_search) / len(bst_search),
        "Red-Black Tree Average Search Iterations": sum(rb_search) /␣
 ↪len(rb_search),
    })

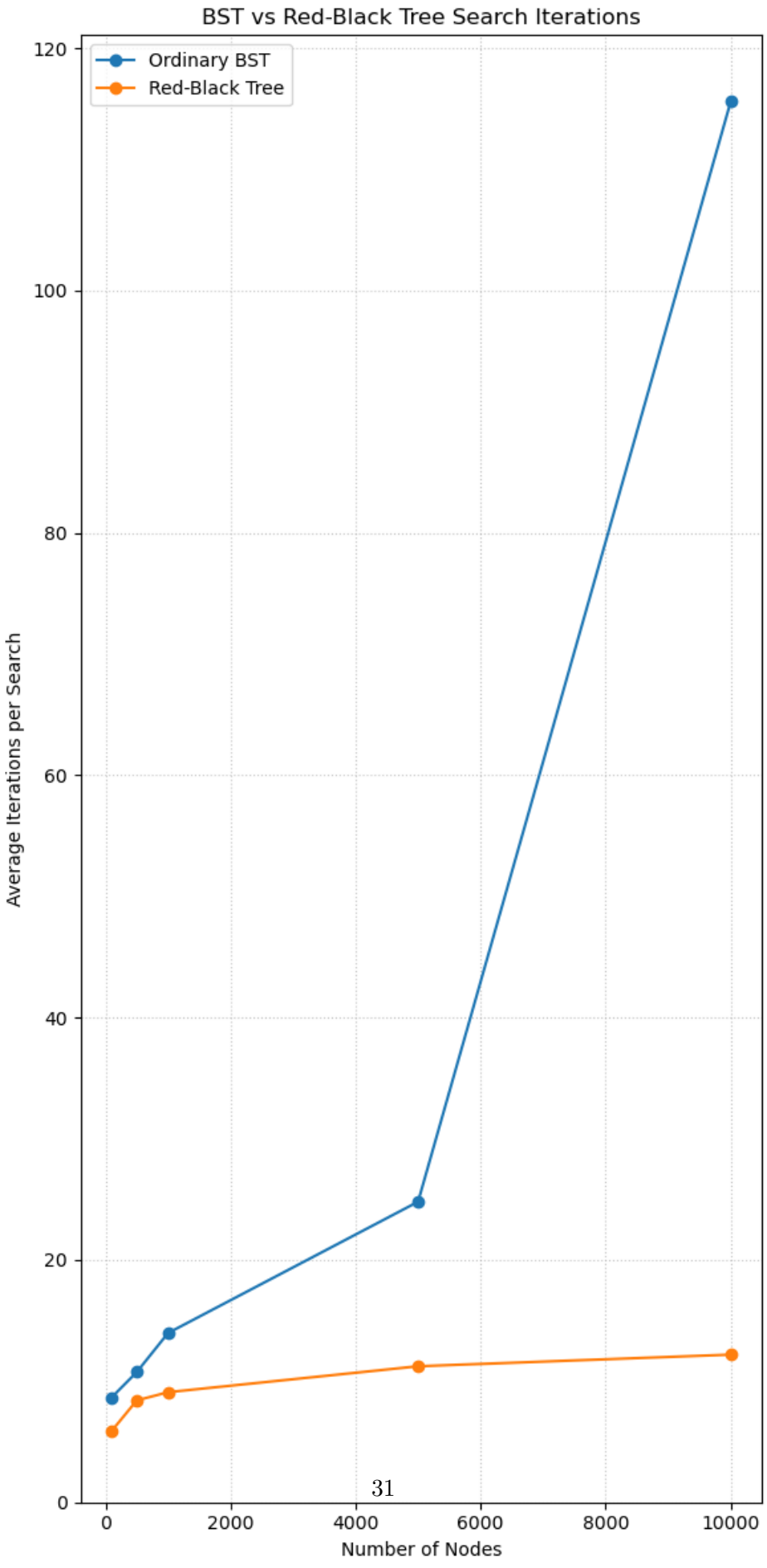
task7_search_df = pd.DataFrame(task7_search_rows)
task7_search_df
```

[30]:

	Nodes	BST Average Search Iterations	Red-Black Tree Average Search Iterations
0	100	8.67	5.90
1	500	10.75	8.41
2	1000	13.97	9.09
3	5000	24.81	11.23
4	10000	115.60	12.19

[32]:
```
plt.figure(figsize=(6, 12))
plt.plot(task7_search_df["Nodes"], task7_search_df["BST Average Search␣
 ↪Iterations"], marker="o", label="Ordinary BST")
plt.plot(task7_search_df["Nodes"], task7_search_df["Red-Black Tree Average␣
 ↪Search Iterations"], marker="o", label="Red-Black Tree")
```

```
plt.xlabel("Number of Nodes")
plt.ylabel("Average Iterations per Search")
plt.ylim(bottom=0)
plt.title("BST vs Red-Black Tree Search Iterations")
plt.legend()
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task7_search.png", dpi=150,␣
  ↪bbox_inches="tight")
plt.show()
```


BST vs Red-Black Tree Search Iterations
Ordinary BST
Red-Black Tree
Average Iterations per Search
Number of Nodes
0
20
40
60
80
100
120
0
2000
4000
6000
8000
10000

5 Q8

Experiment using random, sorted, and reverse-sorted input data and provide a brief analysis of the results.

[ ]:
```
sorted_values = sorted(random_values)
input_orders = {
    "Sorted": sorted_values,
    "Reverse Sorted": list(reversed(sorted_values)),
}
for order_name, values in input_orders.items():
    records[(order_name, "BST")] = BST_Recorded(values, search_idx, delete_keys)
    records[(order_name, "Red-Black Tree")] = RB_Recorded(values, search_idx,␣
 ↪delete_keys)
```

[35]:
```
task8_rows = []
for order_name in ["Random", "Sorted", "Reverse Sorted"]:
    for tree_name in ["BST", "Red-Black Tree"]:
        rec = records[(order_name, tree_name)]
        task8_rows.append({
            "Nodes": N,
            "Input Order": order_name,
            "Tree": tree_name,
            "Height": rec["Height"][N - 1],
            "Insertion Iterations": size_mean(rec["Insertion"], N),
            "Deletion Iterations": size_mean(rec["Deletion"], N),
            "Search Iterations": size_mean(rec["Search"], N),
        })

task8_df = pd.DataFrame(task8_rows)
print("Values at 10000 nodes; iterations are the mean over the last 10% of the␣
 ↪sizes")
task8_df
```

```
Values at 10000 nodes; iterations are the mean over the last 10% of the sizes
```

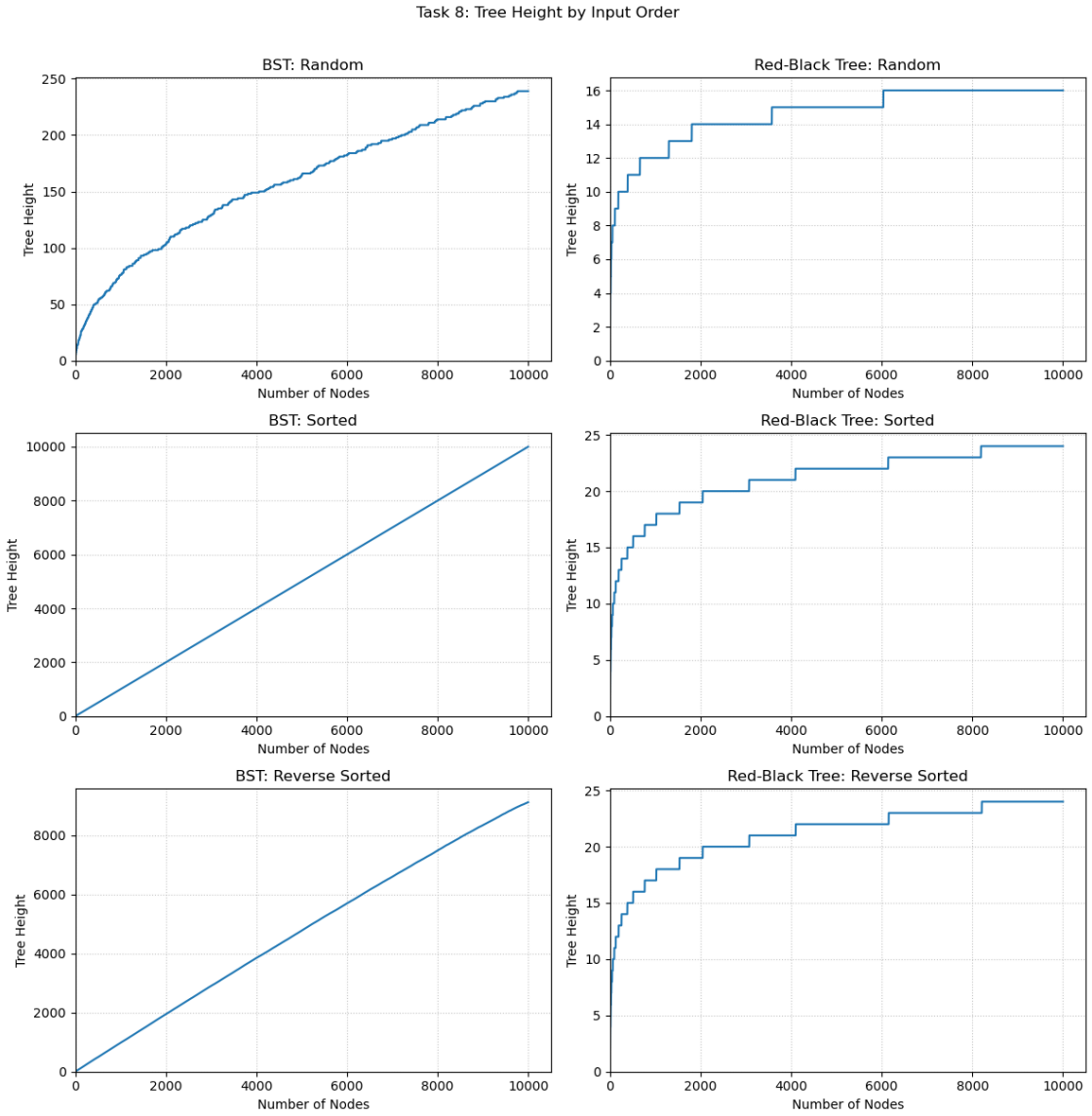
[35]:

	Nodes	Input Order	Tree	Height	Insertion Iterations	Deletion Iterations	Search Iterations
0	10000	Random	BST	239	158.535		
1	10000	Random	Red-Black Tree	16	15.135		
2	10000	Sorted	BST	10000	9499.500		
3	10000	Sorted	Red-Black Tree	24	27.000		
4	10000	Reverse Sorted	BST	9111	8737.645		
5	10000	Reverse Sorted	Red-Black Tree	24	26.815		

0	111.912	109.057
1	14.778	12.243
2	4675.131	4781.168
3	16.804	12.599
4	4563.527	4517.848
5	16.792	12.658

5.1 Height Comparison

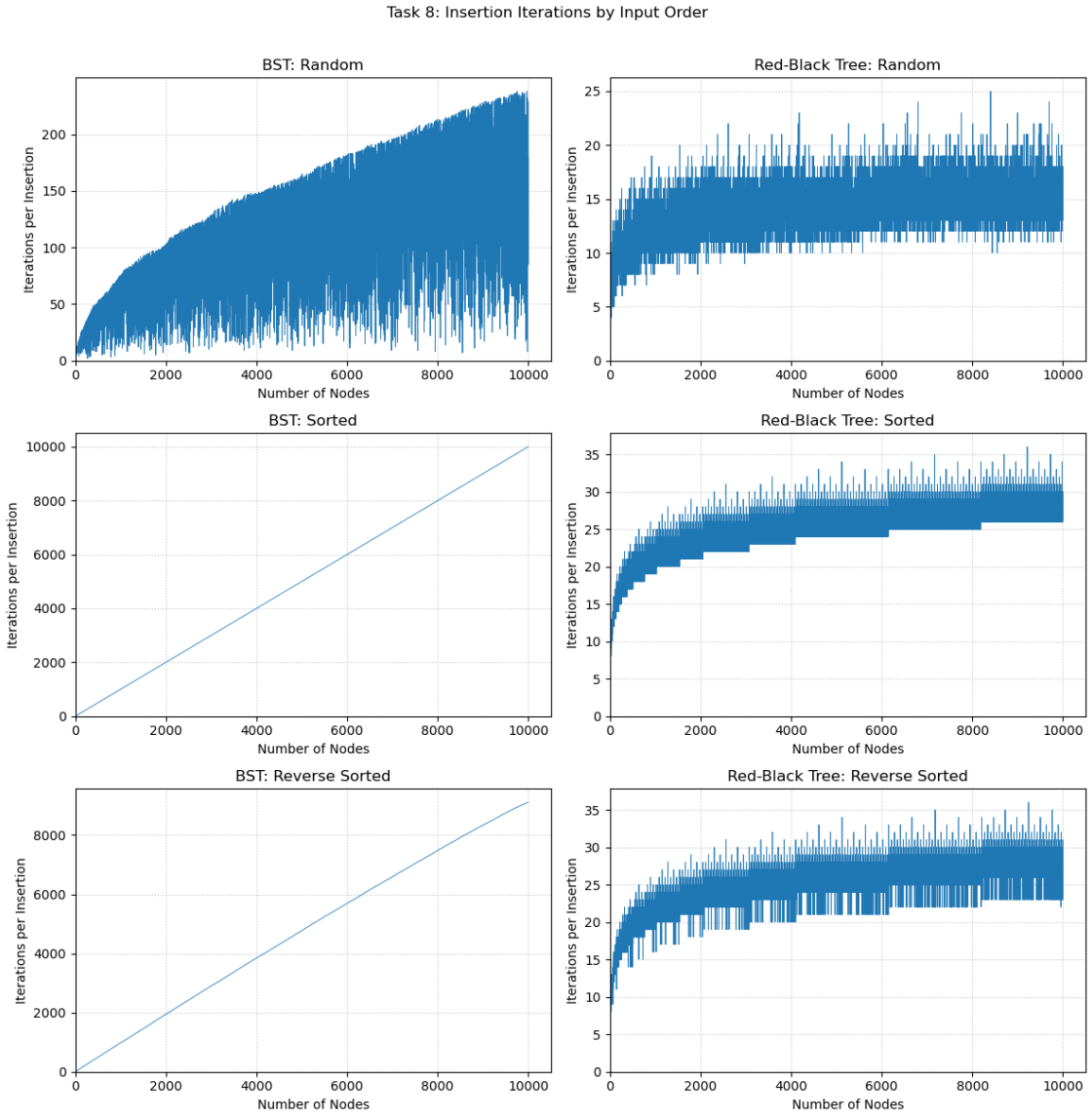
[36]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        y = records[(order_name, tree_name)]["Height"]
        x = [i+1 for i in range(len(y))]
        ax.plot(x, y)
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
        ax.set_ylabel("Tree Height")
        ax.set_xlim(left=0)
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Tree Height by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_height.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



5.2 Insertion Comparison

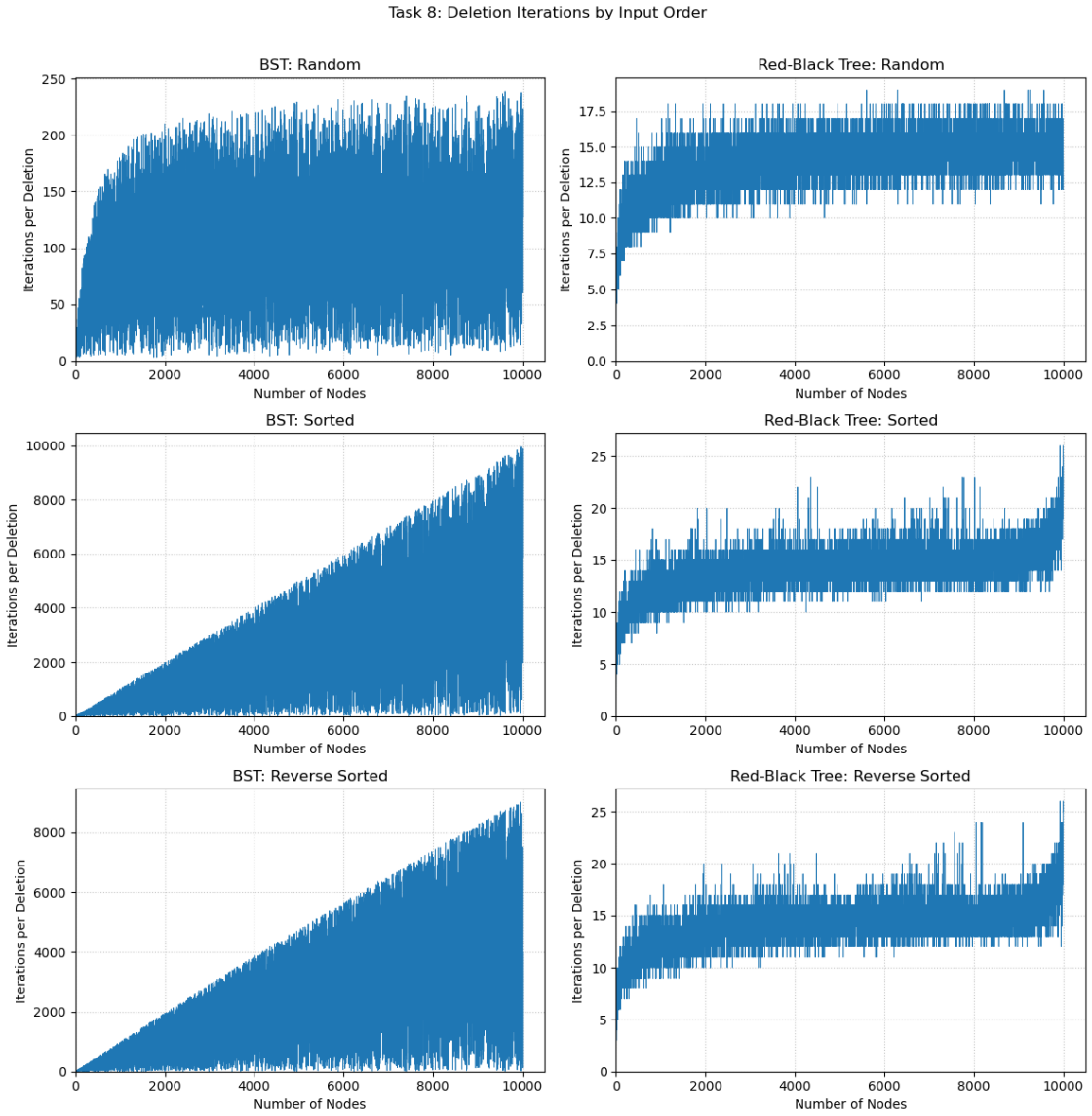
[37]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        y = records[(order_name, tree_name)]["Insertion"]
        x = [i+1 for i in range(len(y))]
        ax.plot(x, y, linewidth=0.6)
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
        ax.set_ylabel("Iterations per Insertion")
```

```
        ax.set_xlim(left=0)
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Insertion Iterations by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_insertion.png", dpi=150,␣
  ↪bbox_inches="tight")
plt.show()
```



5.3 Deletion Comparison

[38]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        y = records[(order_name, tree_name)]["Deletion"]
        x = [i+1 for i in range(len(y))]
        ax.plot(x, y, linewidth=0.6)
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
        ax.set_ylabel("Iterations per Deletion")
        ax.set_xlim(left=0)
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Deletion Iterations by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_deletion.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



5.4 Search Comparison

[39]:
```
task8_search_rows = []
for n in SIZES:
    assert n <= len(arr1), "n cannot exceed the length of arr1"
    start_idx = random.randint(0, len(arr1) - n)
    random_window = arr1[start_idx : start_idx + n]
    sorted_window = sorted(random_window)
    window_orders = {
        "Random": random_window,
        "Sorted": sorted_window,
        "Reverse Sorted": list(reversed(sorted_window)),
```

```
    }
    search_keys = random.sample(sorted_window, min(100, n))

    for order_name, values in window_orders.items():
        for tree_name in ["BST", "Red-Black Tree"]:
            if tree_name == "BST":
                root, _ = BST_Build_Measured(values)
                search_counts = [BST_Search_Measured(root, key)[1] for key in␣
↪search_keys]
            else:
                root, _ = RB_Build_Measured(values)
                RB_Verify(root)
                search_counts = [RB_Search_Measured(root, key)[1] for key in␣
↪search_keys]

            task8_search_rows.append({
                "Nodes": n,
                "Input Order": order_name,
                "Tree": tree_name,
                "Average Search Iterations": sum(search_counts) /␣
↪len(search_counts),
            })

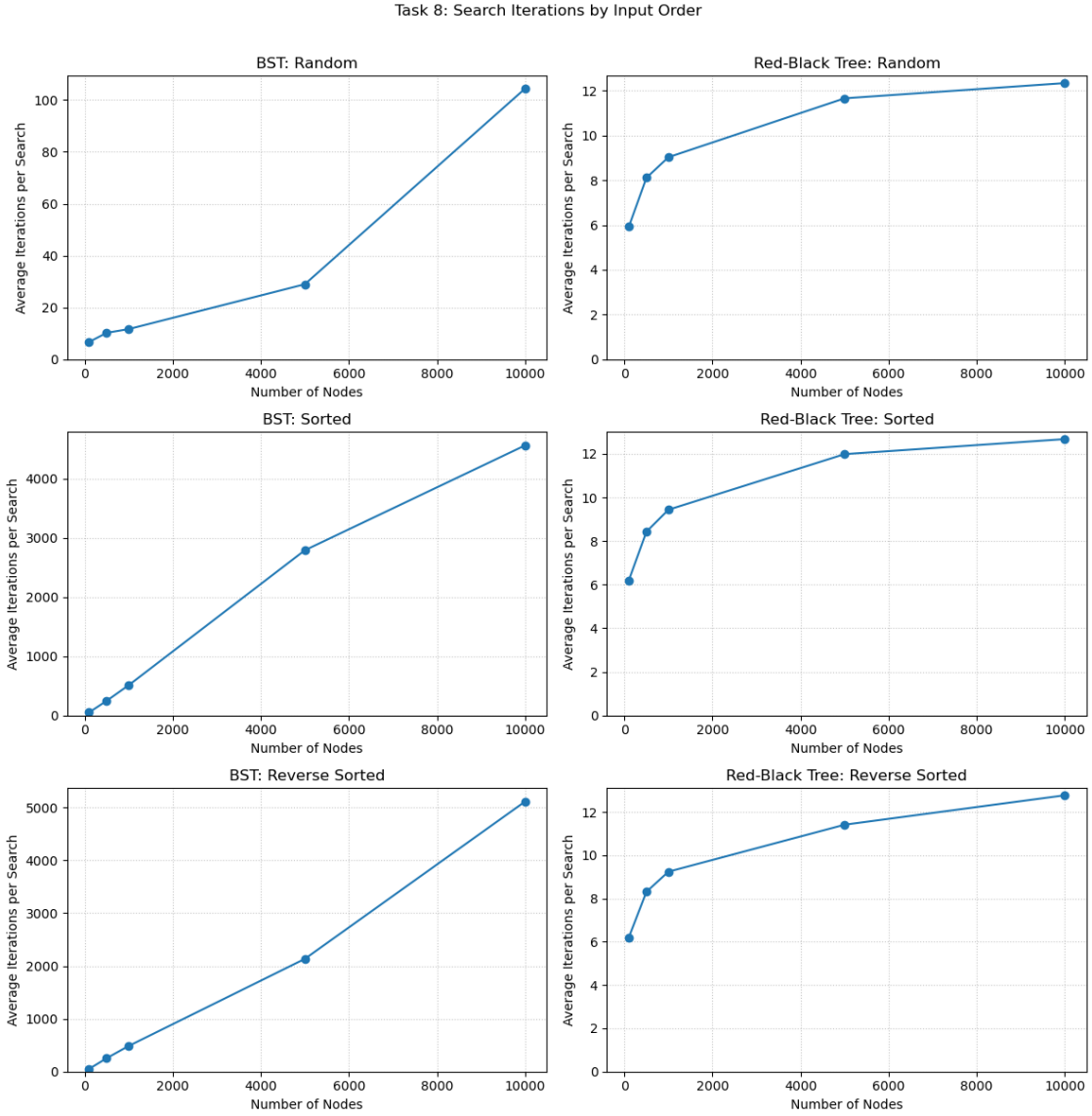
task8_search_df = pd.DataFrame(task8_search_rows)
task8_search_df
```

[39]:

	Nodes	Input Order	Tree	Average Search Iterations
0	100	Random	BST	6.68
1	100	Random	Red-Black Tree	5.92
2	100	Sorted	BST	50.50
3	100	Sorted	Red-Black Tree	6.20
4	100	Reverse Sorted	BST	50.50
5	100	Reverse Sorted	Red-Black Tree	6.20
6	500	Random	BST	10.21
7	500	Random	Red-Black Tree	8.11
8	500	Sorted	BST	247.00
9	500	Sorted	Red-Black Tree	8.43
10	500	Reverse Sorted	BST	252.86
11	500	Reverse Sorted	Red-Black Tree	8.31
12	1000	Random	BST	11.66
13	1000	Random	Red-Black Tree	9.03
14	1000	Sorted	BST	510.61
15	1000	Sorted	Red-Black Tree	9.44
16	1000	Reverse Sorted	BST	485.17
17	1000	Reverse Sorted	Red-Black Tree	9.24
18	5000	Random	BST	28.96
19	5000	Random	Red-Black Tree	11.66

```
20   5000          Sorted             BST                    2790.61
21   5000          Sorted  Red-Black Tree                      11.99
22   5000  Reverse Sorted             BST                    2134.11
23   5000  Reverse Sorted  Red-Black Tree                      11.41
24  10000          Random             BST                     104.42
25  10000          Random  Red-Black Tree                      12.34
26  10000          Sorted             BST                    4561.98
27  10000          Sorted  Red-Black Tree                      12.68
28  10000  Reverse Sorted             BST                    5112.50
29  10000  Reverse Sorted  Red-Black Tree                      12.77
```

[40]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        subset = task8_search_df[(task8_search_df["Input Order"] == order_name)␣
 ↪& (task8_search_df["Tree"] == tree_name)]
        ax.plot(subset["Nodes"], subset["Average Search Iterations"],␣
 ↪marker="o")
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
        ax.set_ylabel("Average Iterations per Search")
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Search Iterations by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_search.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



Input order	Tree	Height	Insertion	Deletion	Search (per size)	Search (100 keys, own tree)
Random	BST	235	161.0	117.0	112.1	107.5
Random	Red-Black	17	15.3	14.9	12.3	12.6
Sorted	BST	10000	9499.5	4700.2	4781.1	4562.0
Sorted	Red-Black	24	27.0	16.8	12.6	12.6
Reverse sorted	BST	9089	8700.9	4530.9	4504.1	5094.9
Reverse sorted	Red-Black	24	26.8	16.8	12.6	12.7